MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Fast and Scalable Diffusion Inference for Text and Video

03/12/26
Ethan Beaird, Blake Potvin,
Kidus Simegne, Torence Mwindaare

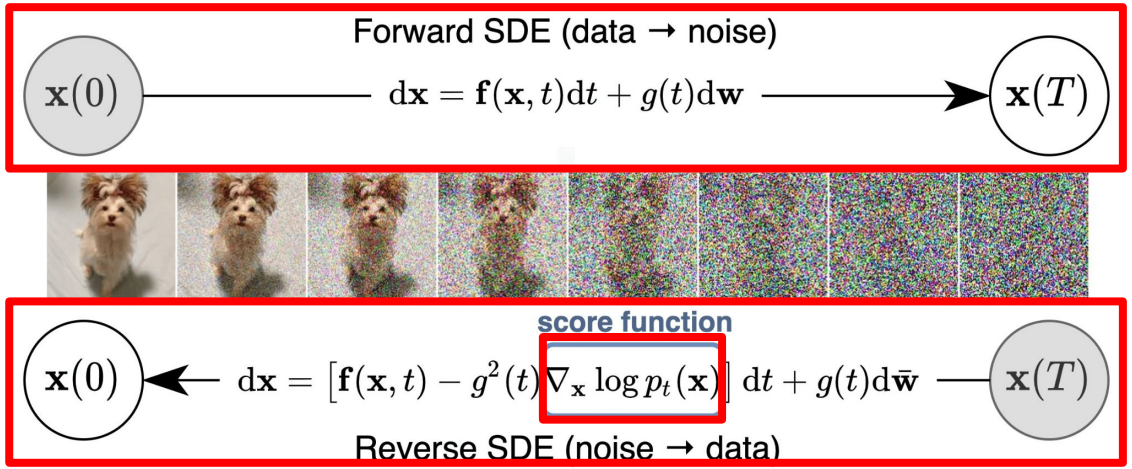
MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Fast-dLLM

TRAINING-FREE ACCELERATION OF DIFFUSION LLM BY Enabling KV CACHE AND PARALLEL DECODING

Diffusion Models

- Fundamental working principle:
  - **Forward process**: adds noise until the data is destroyed.
  - **Reverse process**: neural network learns to remove noise to generate data.



Diffusion on Continuous Vs Discrete Data

- **Continuous diffusion models:**
  - Work with real-valued data.
  - Like images, audio, or sensor readings.
  - Values exist on a **smooth spectrum**.
  - Random **Gaussian noise** added to pixels.

- **Discrete diffusion models:**
  - Data with **distinct**, separate states.
  - Such as text tokens, categorical variables, or binary values.
  - Use **Markov's Chain** of diffusion steps for the forward "noising" (masking) process.

Diffusion based LLMs

- State-of-the-art LLMs **/ Autoregressive (AR)** models predict next word based on previous word
    - Use **unidirectional** attention (often from left to right)
- Diffusion models utilize parallel (**Non-autoregressive)** token generation and **bidirectional** attention mechanism
    - It starts from a fully “noisy” sequence representation
    - Progressively refines it, removing masking through multiple steps to arrive at the final text.
- Unlike AR models that generate tokens sequentially, the ability of diffusion models to generate **multiple tokens** or the entire sequence **simultaneously**, potentially leading to speed advantages.
- They refine outputs iteratively, leading to higher quality and better control over the final structure.

Masked Diffusion Models: Forward Process

- Forward noising process via a discrete Markov chain.
- Tokens are progressively replaced by a special **[MASK]** token.
- Transition probability defining this noising process:

$$q_{t|0}\left(\boldsymbol{x}_t|\boldsymbol{x}_0\right)=\prod_{i=1}^{n} q_{t|0}\left(\boldsymbol{x}_t^i|\boldsymbol{x}_0^i\right)=\prod_{i=1}^{n}\mathrm{Cat}\left(\boldsymbol{x}_t^i;(1-t)\delta_{\boldsymbol{x}_0^i}+t\delta_{[\mathrm{MASK}]}\right).$$

- Masking level, $t \in [0, 1]$
- original data $x0$ and fully masked sequence at t = 1

Masked Diffusion Models: Reverse Process (τ -leaping)

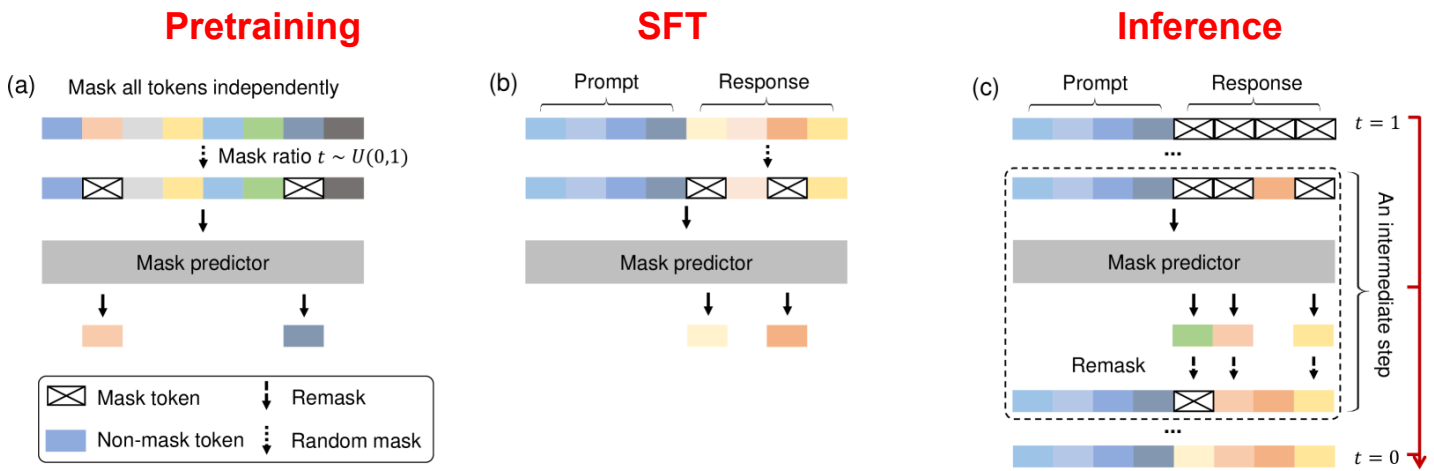
- Analytical reverse of the forward process is computationally inefficient for generation
- Employ τ -leaping approximation:

$$q_{s|t} = \prod_{i=0}^{n-1} q_{s|t}(\boldsymbol{x}_s^i|\boldsymbol{x}_t), \text{ where } q_{s|t}(\boldsymbol{x}_s^i|\boldsymbol{x}_t) = \begin{cases} 1, & \boldsymbol{x}_t^i \neq [\text{MASK}], \boldsymbol{x}_s^i = \boldsymbol{x}_t^i \\ \frac{s}{t}, & \boldsymbol{x}_t^i = [\text{MASK}], \boldsymbol{x}_s^i = [\text{MASK}] \\ \frac{t-s}{t} q_{0|t}(\boldsymbol{x}_s^i|\boldsymbol{x}_t), & \boldsymbol{x}_t^i = [\text{MASK}], \boldsymbol{x}_s^i \neq [\text{MASK}]. \end{cases}$$

Figure adapted from *LLaDA: Large Language Diffusion Models*. (2024)

LLaDA: A Diffusion-Based Language Model

- LLaDA employs a Transformer as the **mask predictor**, whose architecture is similar to existing LLMs.
- However, LLaDA does not use a **causal mask**, as its formulation allows it to see the entire input for predictions.



Challenges with Diffusion LLMs

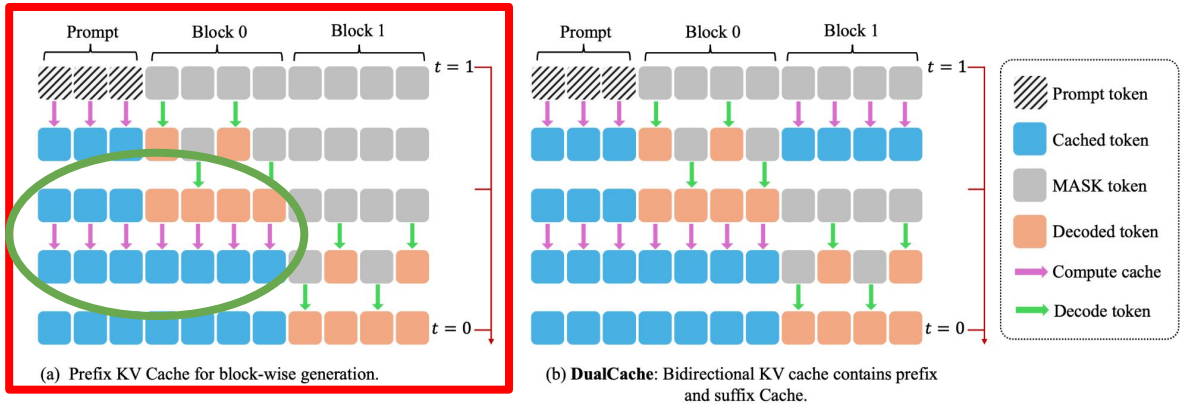
- Actual speed of current open-source diffusion LLMs falls behind AR models
- This is primarily due to:
    - Lack of efficient KV caching compared to AR models (**causal attention**)
    - **Full bidirectional attention**: every token attends to all others
- Generation quality of diffusion models degrades when decoding multiple tokens all at once

Contributions of Fast-dLLM

- **Problem**: Absence of key-value (KV) caching for Non-autoregressive models unlike AR models
- **Solution:**
  - **Key-Value Cache for Block-Wise Decoding:** They introduce a block-wise approximate KV Cache mechanism specifically designed for bidirectional attention.
- **Problem:** Degradation in generation quality when decoding multiple tokens.
- **Solution:**
  - **Confidence-Aware Parallel Decoding:** Instead of decoding fixed, multiple tokens per step, dynamically select tokens based on global confidence threshold.

Methodology: KV Cache for Block-Wise Decoding (Prefix Only Caching)

- **Blockwise** decoding to support use of KV cache
- Compute KV cache for the prompt
- **Reuse** the cache inside a block across multiple steps
- After a block is finalized, the KV cache is updated with the newly generated tokens.

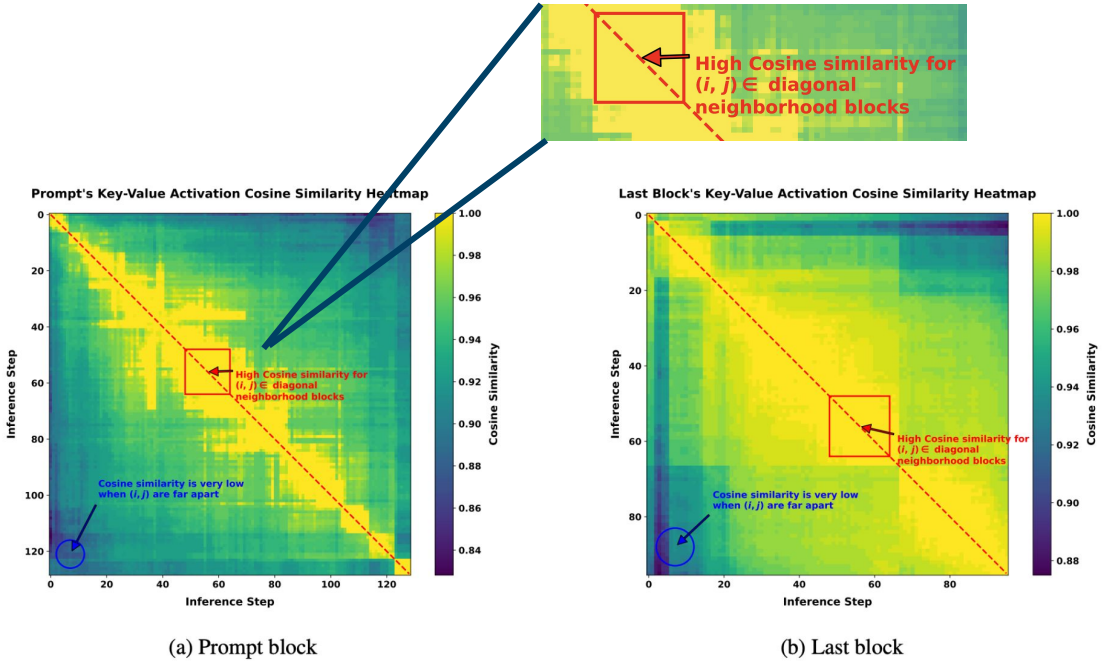


(a) Prefix KV Cache for block-wise generation.

(b) **DualCache**: Bidirectional KV cache contains prefix and suffix Cache.

KV Activation Similarity Across Steps

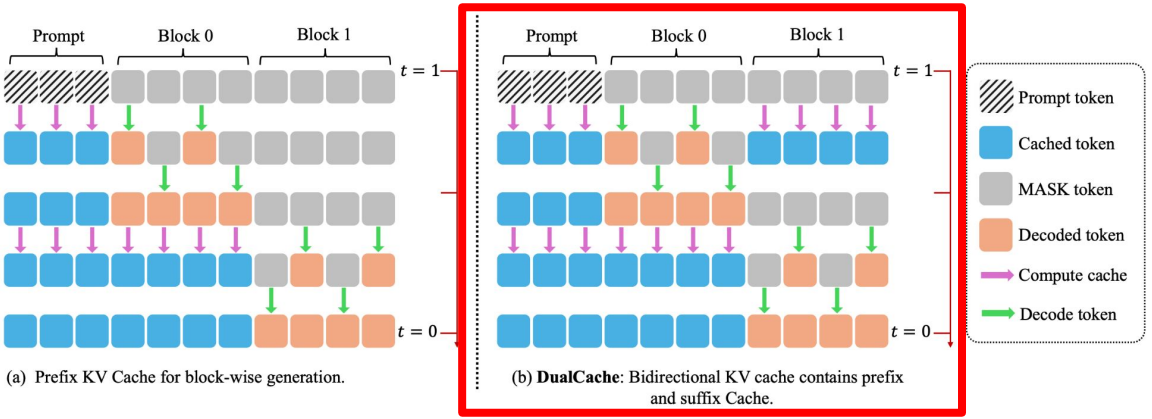
- Justification for KV cache reuse across multiple steps
- Activations across adjacent denoising (inference) steps:
  - **KV(step i) ≈ KV(step j)**
  - Given that i and j are diagonal neighborhood blocks
- This is true for both suffix and prefix keys and values.
- Safe cache reuse with **minimal** approximation error.



(a) Prompt block (b) Last block

Methodology: KV Cache for Block-Wise Decoding (Bidirectional)

- Implement **DualCaching**
- Also cache the **suffix tokens**
- Suffix token consists entirely of masked tokens at first

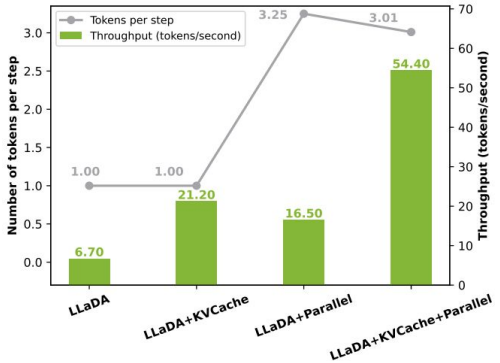


(a) Prefix KV Cache for block-wise generation.

(b) **DualCache**: Bidirectional KV cache contains prefix and suffix Cache.

Applying Approximate KV-Caching on LLaDA

- Baseline non-autoregressive **diffusion based LLM**
- Lacks the approximate blockwise KV cache and confidence aware parallel decoding technique **Fast-dLLM** introduced
- As a result, it takes significant performance hit compared to it's Fast-dLLM-optimized version.



Why naive parallel decoding breaks

- Reversing this is slow: $$q_{t|0}\left(\boldsymbol{x}_t|\boldsymbol{x}_0\right) = \prod_{i=1}^{n} q_{t|0}\left(\boldsymbol{x}_t^i|\boldsymbol{x}_0^i\right) = \prod_{i=1}^{n} \mathrm{Cat}\left(\boldsymbol{x}_t^i; (1-t)\delta_{\boldsymbol{x}_0^i} + t\delta_{[\mathrm{MASK}]}\right).$$

- So we use τ-leaping: $q_{s|t} = \prod_{i=0}^{n-1} q_{s|t}(\boldsymbol{x}_s^i|\boldsymbol{x}_t)$ and multiple masked tokens will be generated in parallel.

Why naive parallel decoding breaks

- But we assume **conditional independence:** We sample from $p(\boldsymbol{x}_s^i|\boldsymbol{x}_t) \cdot p(\boldsymbol{x}_s^j|\boldsymbol{x}_t)$ but the true joint probability requires accounting for the dependency: $p(\boldsymbol{x}_s^i, \boldsymbol{x}_s^j|\boldsymbol{x}_t) = p(\boldsymbol{x}_s^i|\boldsymbol{x}_t) \cdot p(\boldsymbol{x}_s^j|\boldsymbol{x}_t, \boldsymbol{x}_s^i)$

- **Example:** *The list of poker hands that consist of two English words are: __ __.*
  - *'high card', 'two pair', 'full house', 'straight flush'* → ***'high house'.***

Methodology: Confidence-Aware Parallel Decoding

- **<u>Idea:</u> Decode high-confidence tokens now; defer the uncertain ones until later.**
- At each step, compute a *confidence score* (maximum predicted probability over the vocabulary) for every masked token. Only unmask tokens whose confidence is above a threshold.

$$\text{For masked } x^i, \text{ compute confidence } c^i = \max_x p_{\boldsymbol{\theta}}(x^i|\cdot)$$

How is this even justifiable?

- **When is it theoretically justifiable to decode tokens in parallel using independent margins, despite the true joint distribution potentially containing dependencies?**

- **Theorem 1** (in layman's terms)**:** If every token in some candidate sequence has **very high marginal confidence** (each token has probability > $1-\epsilon$, and $(n+1)\epsilon \leq 1$ for n parallel tokens):
    - **Greedy parallel decoding** yields the same result as **greedy sequential decoding**

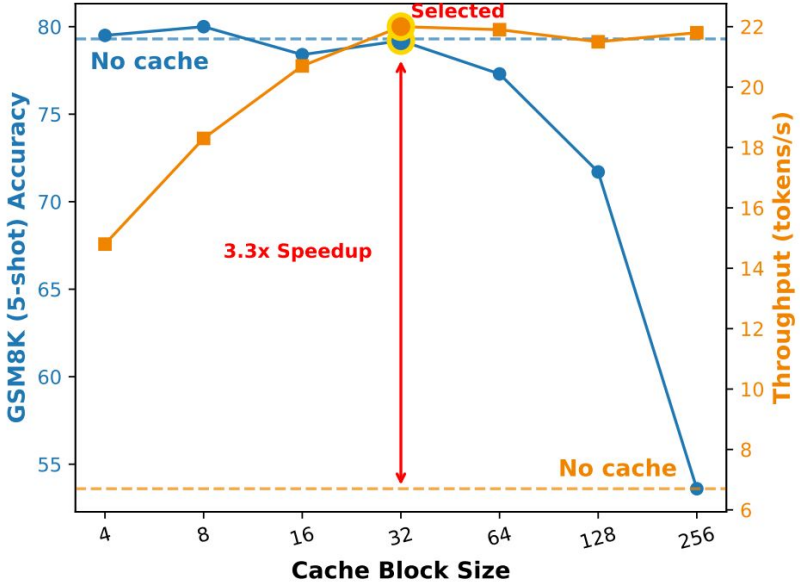
How is this even justifiable?

- **Factor-based parallel decoding strategy:** Given the model's marginal confidence estimate for n tokens in a block, sort these confidences and select the largest n s.t. $(n + 1)(1 - c^{(n)}) < f$ where f is a fixed decoding factor hyperparam and $c^{(n)}$ is the n-th highest confidence.

- At each step, the top-n tokens are decoded in parallel.

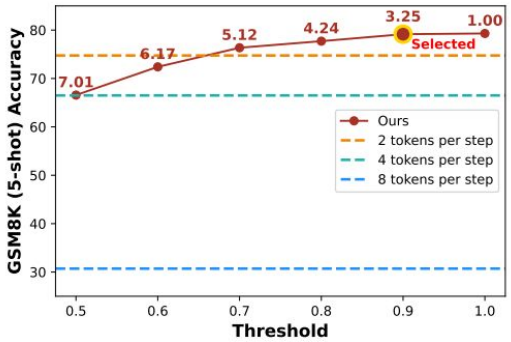
Experimental Setup

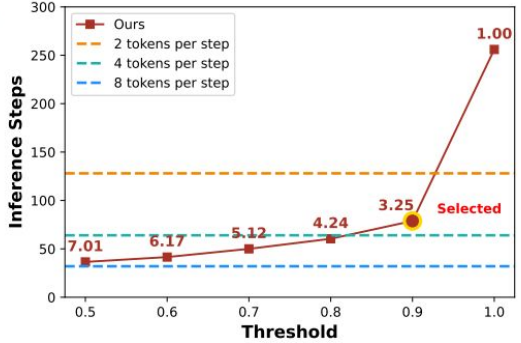
- Tested on NVIDIA A100 80GB GPU
- Evaluated on diffusion-based language models
  - **LLaDA, LLaDA-1.5, Dream**
- Benchmarked on common datasets to assess performance across reasoning and code generation tasks
  - **GSM8K, MATH, HumanEval, MBPP**
- Also extended to multi-modal tasks
  - LLaDA-V model, MathVista and MathVerse benchmarks

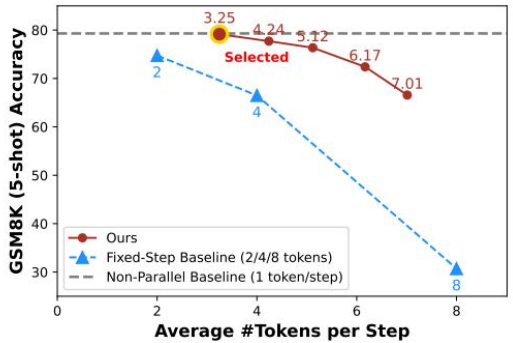


Cost/step ↓ , Number of steps ↓

- **KV Cache mechanism** yields 2-3.6x speedup, **parallel decoding strategy** yields 4-6x speedup
- **Combined** boost throughput up by 9.2-11x
- Efficiency gains achieved with **negligible impact on accuracy**
- **Longer sequences benefit** proportionally more from caching and parallelization techniques due to greater cache reuse and batch computation.







Multi-Modal version (LLaDA-V)

- LLaDA-V is highly sensitive to block size (accuracy drops as block size lowers)
- Retain a **full block length** and apply refresh-based updates instead of small-block caching.
- **9.9x speedup** with minimal accuracy degradation
  - Accuracy is even slightly improved on MathVerse
- Improvements generalize across model architectures, task types, and modalities
  - **Fast-dLLM** is practical and broadly applicable for accelerating masked diffusion LMs.

Table 9 | Effect of block length on performance (MathVista, 48 Steps)

Block Length	4	8	16	32	96
Accuracy (%)	51.2	50.7	51.8	52.3	59.7
Throughput (tok./s)	6.1	6.2	5.5	5.5	5.6

What drives the speedup?

- **Ablation highlights: Acceleration Behavior**
- Longer prompts and longer generations help more.

Setting.	LLaDA	Parallel Decoding		
		No Cache	PrefixCache	DualCache
5-shot	77.0 1.1 (1×)	77.4 11.7 (10.6×)	75.2 14.4 (13.1×)	74.7 21.6 (19.6×)
8-shot	77.3 0.7 (1×)	78.0 9.3 (13.3×)	75.7 13.0 (18.6×)	76.0 19.3 (27.6×)

- Caching benefits compound at large scales.

Len.	LLaDA	Parallel Decoding		
		No Cache	PrefixCache	DualCache
256	77.6 4.9 (1×)	77.9 16.4 (3.3×)	77.3 49.2 (10.0×)	76.9 46.3 (9.4×)
512	78.9 2.3 (1×)	78.9 14.0 (6.1×)	74.8 32.0 (13.9×)	75.4 36.4 (15.8×)
1024	77.3 0.7 (1×)	78.0 9.3 (13.3×)	75.7 13.0 (18.6×)	76.0 19.3 (27.6×)

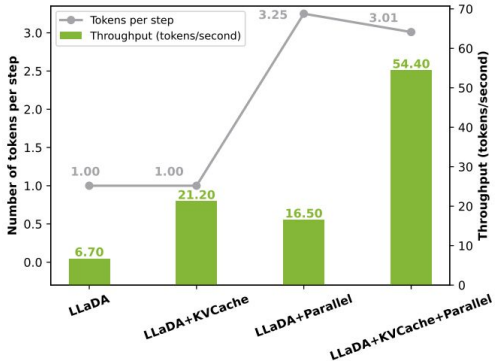
- DualCache is the strongest cache variant.

- **Example result:**
  - At 8-shot, generation length 1024, DualCache reaches 27.6x speedup

Why the decoding strategy works

- **Ablation highlights: quality/speed tradeoff**
- Block size matters.
  - Smaller blocks: more accurate, more overhead
  - Larger blocks: more efficient, hurt quality
  - Best tradeoff: size 32

- Confidence-aware parallel decoding beats fixed-step decoding:
  - Higher accuracy, fewer inference steps



Why the decoding strategy works

- **Ablation highlights: quality/speed tradeoff**
- Dynamic decoding is better than static decoding
  - Avoids waste of decoding conservatively and quality loss of decoding too aggressively

- Factor decoding has similar accuracy to threshold-based decoding
  - But higher throughput

User: Please describe the image in detail.

Baseline (63.0 secs):

Fast-dLLM (6.8 secs):

Fast-dLLM Limitations

- KV cache mechanism is **approximate.**
- Parallel decoding strategy is only reliable when token confidences are sufficiently high.
- Hyperparameter-sensitive. Performance depends heavily on block size and confidence threshold.
- Still doesn’t remove diffusion-model efficiency bottlenecks.
    - At large batch sizes, struggles to match LLaMA because diffusion decoding has full-attention computation.

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

ScaleFusion

SCALABLE INFERENCE OF SPATIAL-TEMPORAL DIFFUSION TRANSFORMERS FOR HIGH-RESOLUTION LONG VIDEO GENERATION

Video Diffusion

Generate a video by iteratively denoising from noise.

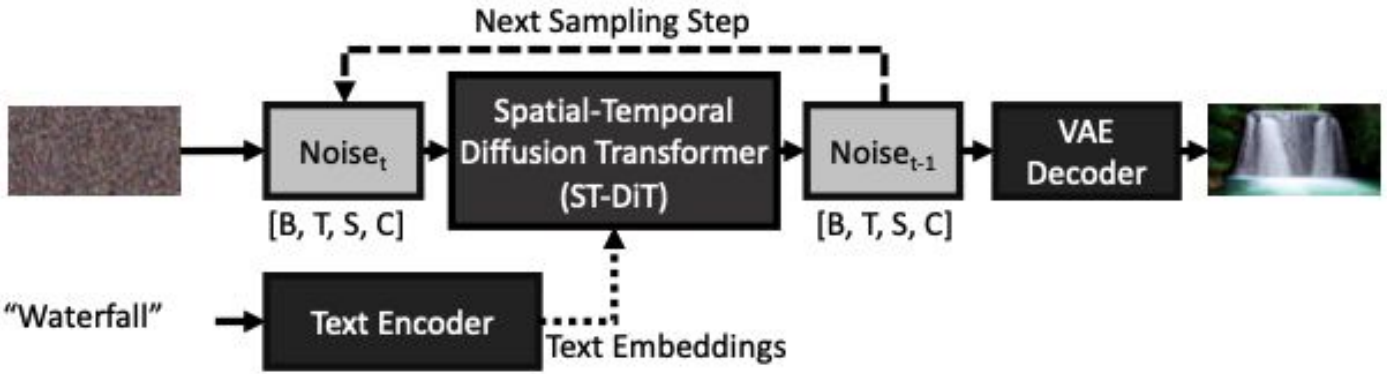
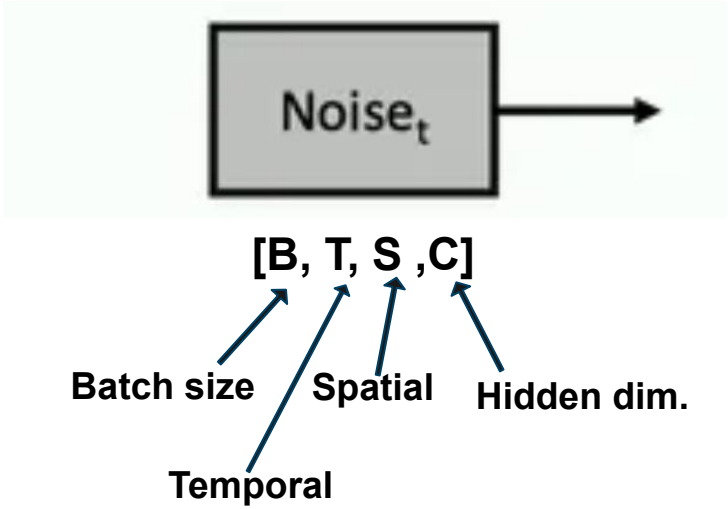


Figure 2. Illustration of video diffusion processes with ST-DiT.

What are **Spatial-Temporal Diffusion Transformers (ST-DiT)?**

- A transformer denoiser for video diffusion that models video tokens across space & time.
- **Example Models**: Open Sora, Tora
- ST-DiT operates on video tokens shaped like [B, T, S, C]:
  - **B** = batch size (number of videos processed together)
  - **T** = temporal length (frames, T= duration x FPS)
  - **S** = spatial tokens per frame (patches; grows with resolution)
  - **C** = hidden dimension (embedding size per token)



How ST-DiT Attention Works (Spatial vs Temporal)

Alternates between:

- **Spatial attention (along S)**:
  - within each frame, attend over all S patch tokens
  - learns objects/layout/within-frame structure
- **Temporal attention(along T)**:
  - across frames for each spatial position s: attend over all T frames
  - enforces motion & consistency (reduces flicker)

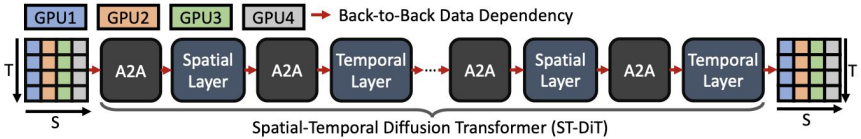


Figure 1. The distributed execution of ST-DiT introduce back-to-back dependencies.

Distributed Inference for ST-DiT

- We shard tokens across **P GPUs** (start **spatial-distributed**: each GPU has **[B, T, S/P, C]**).
  E.g: GPU0 holds the first S/P patch tokens of *every frame*, GPU1 holds the next S/P of *every frame*, and so on.

- Spatial attention needs full **S** then **All-to-All** transforms to **temporal-distributed**: **[B, T/P, S, C]**

- Temporal attention needs full **T** then another **All-to-All** transforms back to **[B, T, S/P, C]**

A2A > compute > A2A > compute…

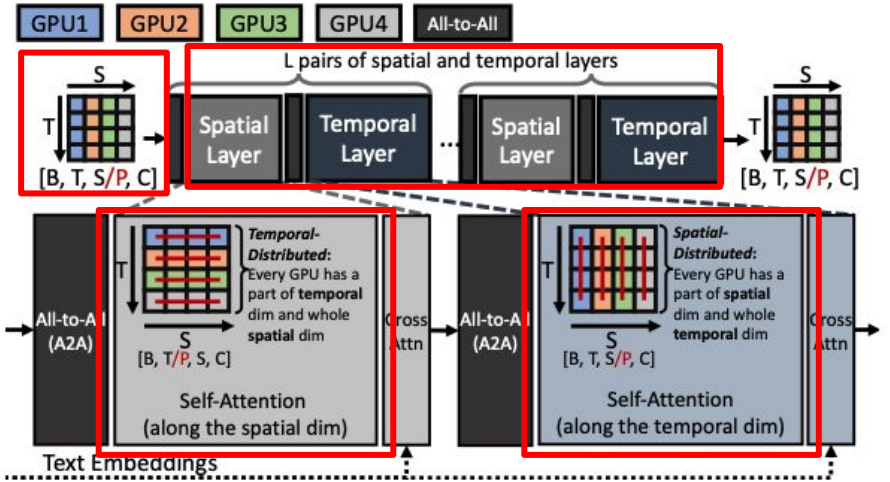


Figure 3. Distributed execution of ST-DiT. For simplicity, the auxiliary layers such as layer normalizations are not shown.

Scaling Problem: back-to-back dependencies

- **Back-to-back dependency** as each stage needs the output of the previous one
- **No overlap:** communication and compute don't run at the same time
- Communication becomes the bottleneck, so adding GPUs gives poor scaling

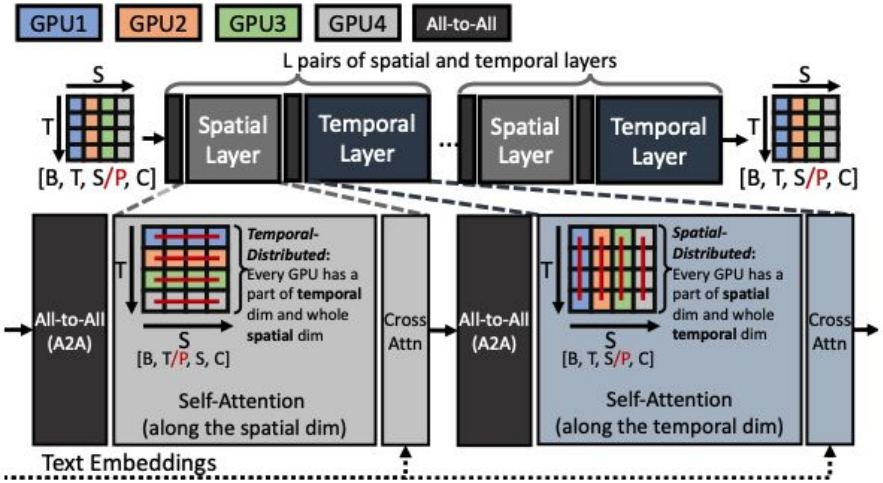
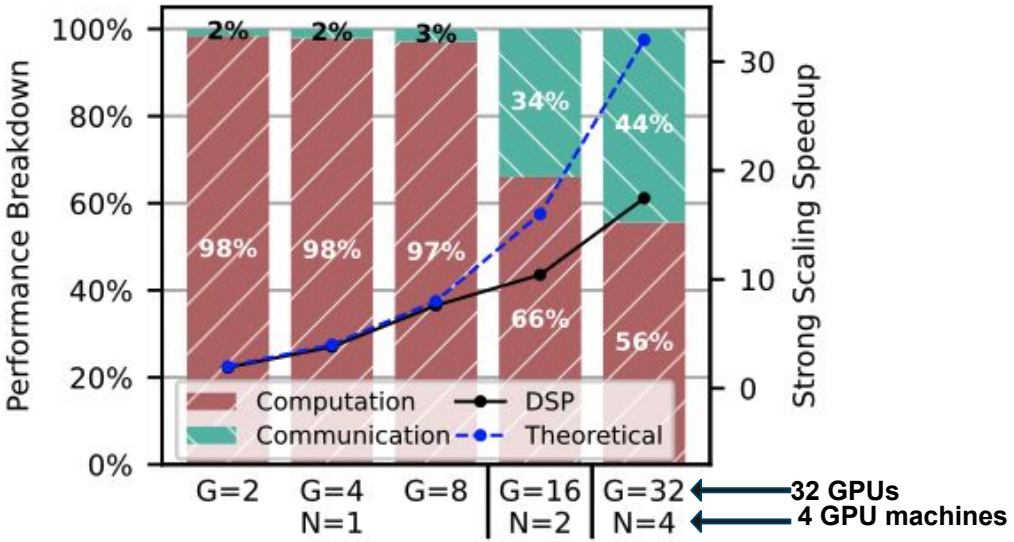


Figure 3. Distributed execution of ST-DiT. For simplicity, the auxiliary layers such as layer normalizations are not shown.

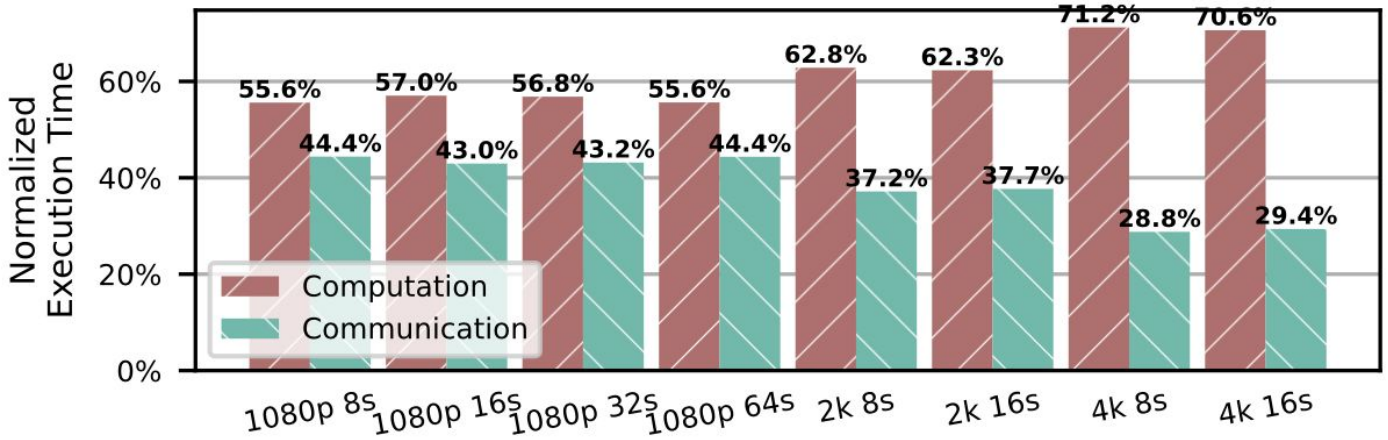
Scaling Problem: back-to-back dependencies

As we increase GPUs/machines, the fraction of time in communication grows, so scaling falls below ideal.



High Resolution, Longer Videos become difficult

- As resolution increases, the workload becomes more compute-heavy but communication remains a significant chunk.
- This motivates need for overlap: we want to hide A2A comm. under compute rather than paying it serially.



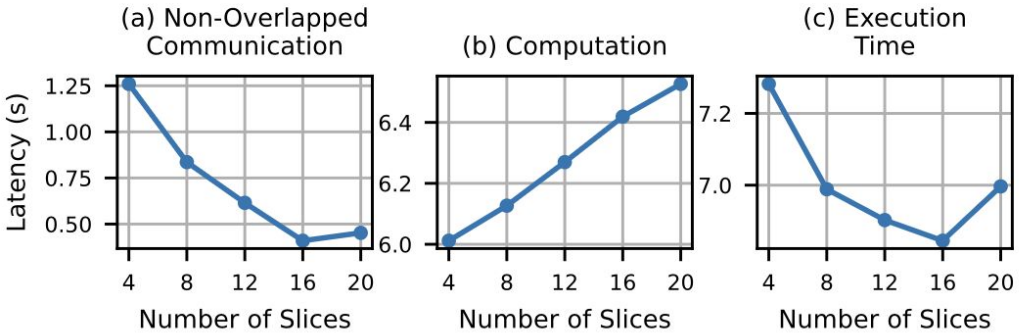
Existing Optimizations for Distributed transformer inference

- **Parallelism**
  - Sequence parallelism (split tokens across GPUs): DeepSpeed-Ulysses, RingAttention
  - Tensor / model parallelism (split weights / heads)
  - Data parallelism (batch more samples)
- **Lossy techniques (image-focused)**
  - Reuse/caching activations across diffusion steps to avoid recompute/comm
  - Tradeoff: can introduce stale features and quality risk (worse for videos)

These are not enough!

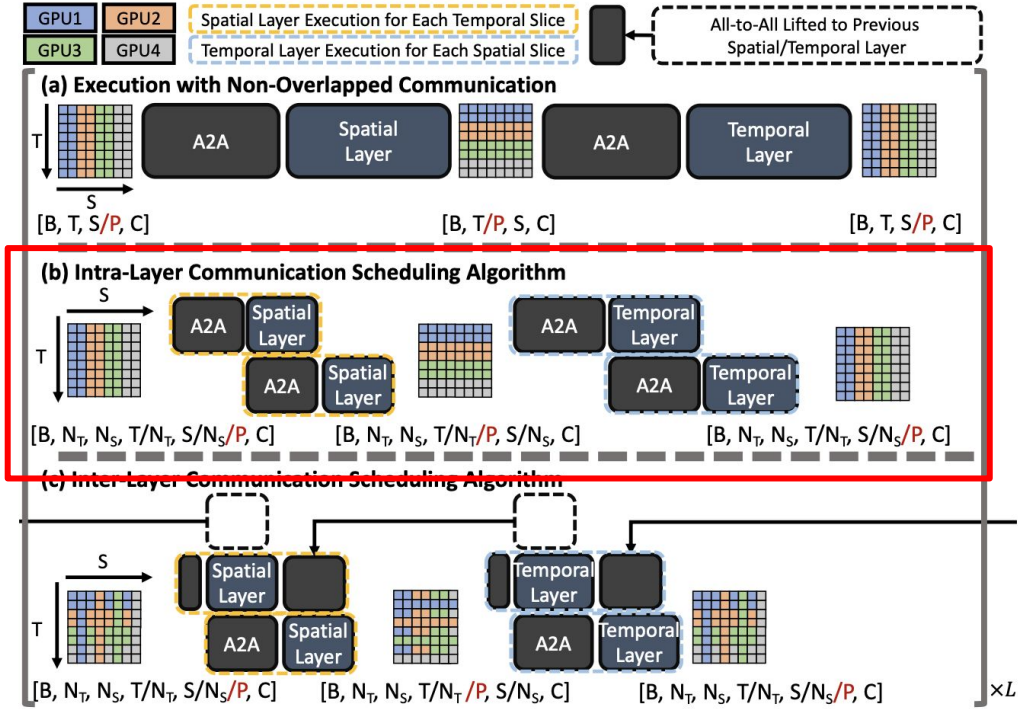
Optimization Challenge: Diminishing speedups with increasing slices

- Split the sequence into slices within a GPU to get partial outputs early.
- **Better Overlap:** start A2A**(slice i)** while computing **slice i+1**
- However, too many slices hurt compute efficiency (more launches, smaller matmuls, less GPU utilization).



Overall time becomes U-shaped. There's an optimal slice count (more is not always better).

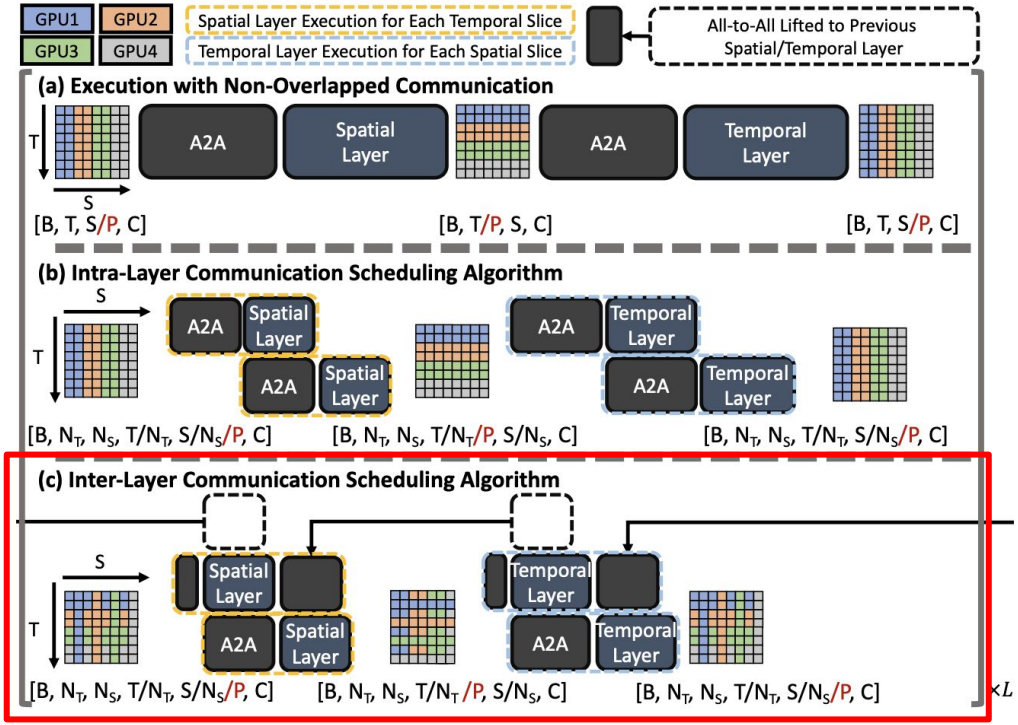
ScaleFusion Overview: Two Scheduling Opportunities



Challenge 1: Back-to-back dependencies (no overlap)

Intra-layer Comm. Scheduling Alg. (Fig b): Hide communication by slicing

ScaleFusion Overview: Two Scheduling Opportunities



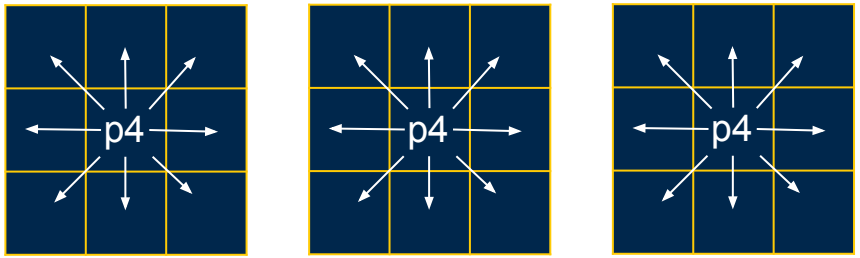
Challenge 2: More slices increases compute overhead

Inter-layer Comm. Scheduling Alg. (Fig b): Hides the remaining bubbles without increasing slices and slowing compute

Spatial-Temporal Independence

Spatial Attention is Independent along T

- Each frame's attention matrix only corresponds to tokens from that frame



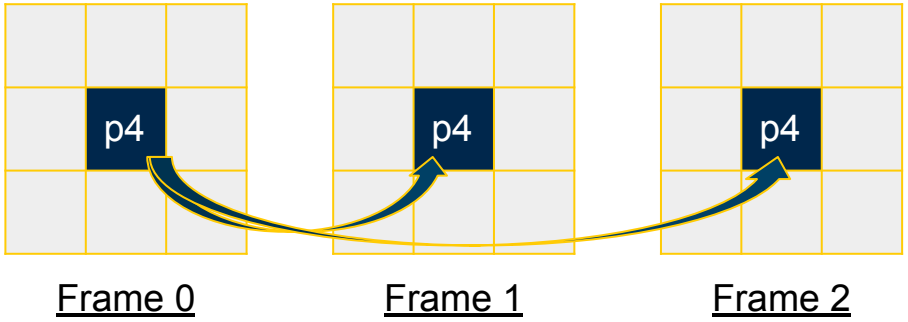
Spatial Attention of patch 4 in each frame
Note: No patch is calculating across a different frame due to independence

Spatial-Temporal Independence

Temporal Attention is Independent along S

- Each spatial patch's attention matrix only contains that position across all frames

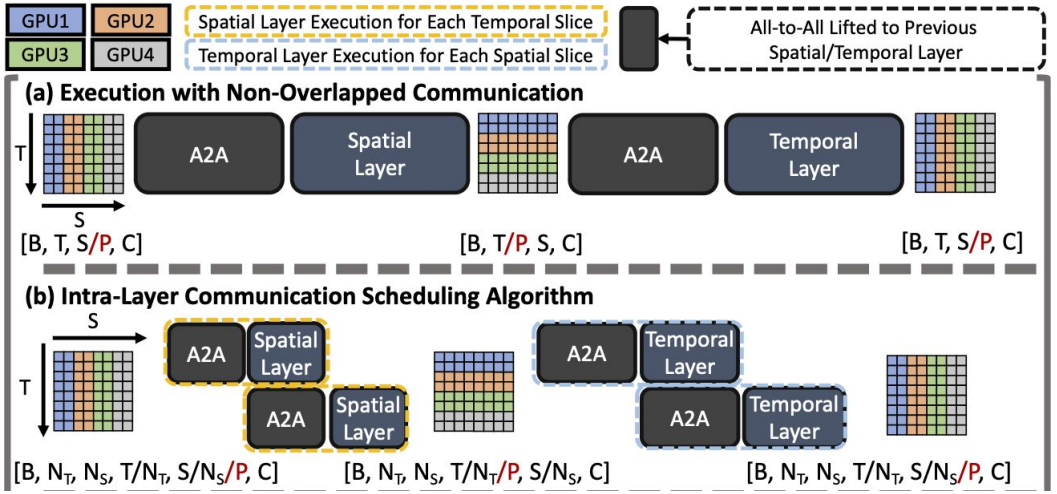
Unlike prior works, this makes the attention computation **Lossless**



Temporal Attention of patch 4 across 3 frames
Note: Patch 4 does NOT use any other patches due to independence

Intra-Layer Communication Scheduling

Key Idea: Hide communication latency from A2A by splitting layer execution into different slices



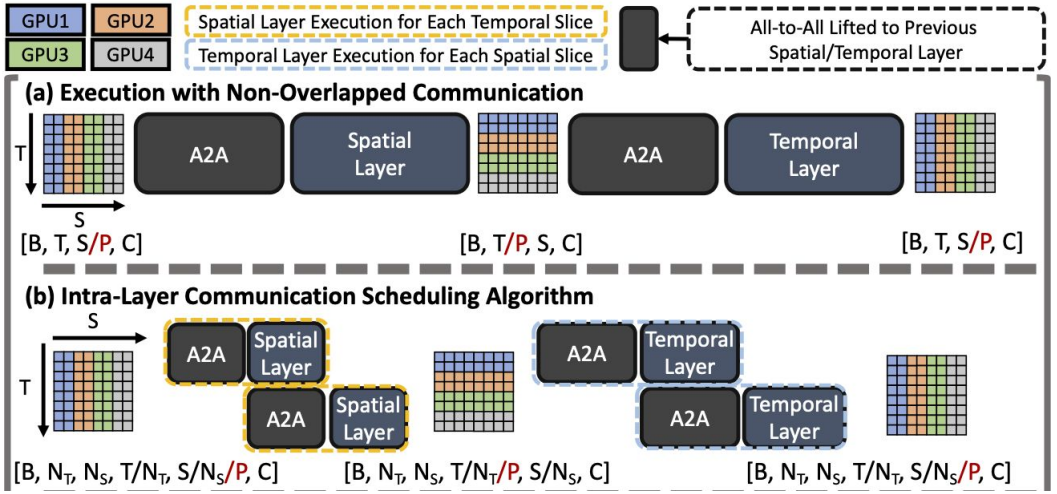
Intra-Layer Communication Scheduling

2 New Hyperparameters:
N_T and N_S

N_T : Number of slices in the temporal dimension

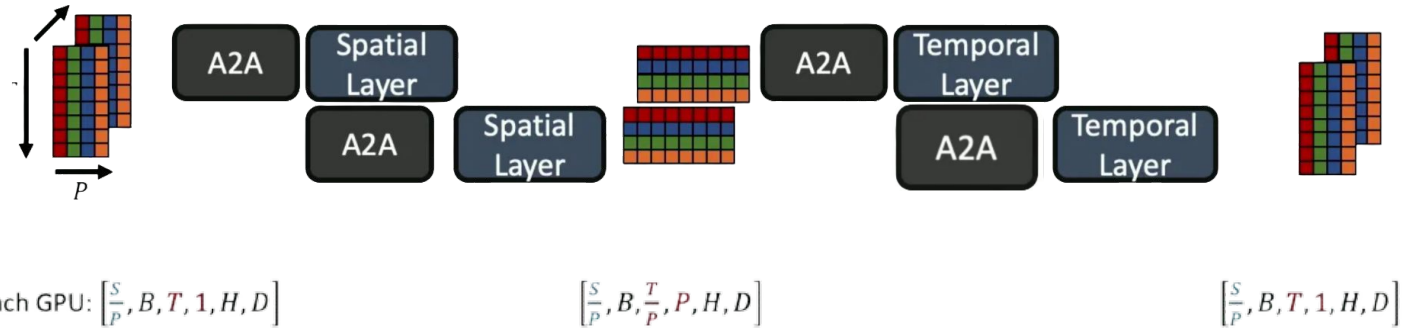
N_S : Number of slices in the spatial dimension

Both are bound to be ≤ T or S respectively



Intra-Layer Communication Scheduling

- Execution starts with an A2A of the first slice on spatial layer
- Followed by a parallel execution of spatial attention and a simultaneous A2A of next slices
- Repeats for each slice and overall repeat for temporal attention



Example Above is $N_T = N_S = 2$

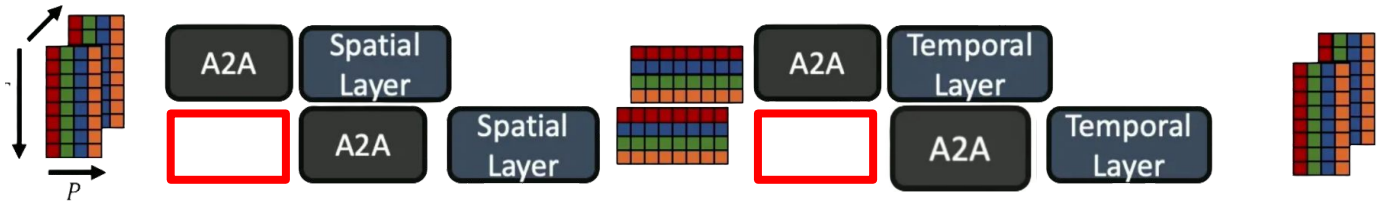
Intra-Layer Communication Scheduling

$Comm_T$ and $Comm_S$ represent communication latency of temporal and spatial layers respectively

Before: $Comm_T + Comm_S$

After: $Comm_T/N_T + Comm_S/N_S$

Problem: How do we resolve the gaps in the initial A2A blocks??

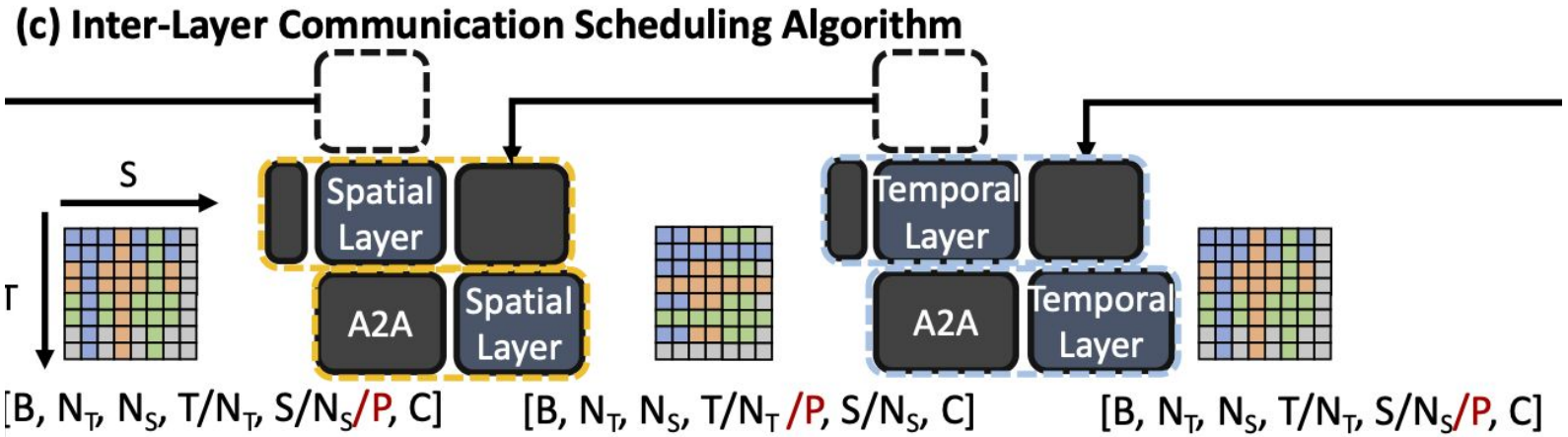


Each GPU: $\left[\frac{S}{P}, B, T, 1, H, D\right]$ $\left[\frac{S}{P}, B, \frac{T}{P}, P, H, D\right]$ $\left[\frac{S}{P}, B, T, 1, H, D\right]$

Example Above is $N_T = N_S = 2$

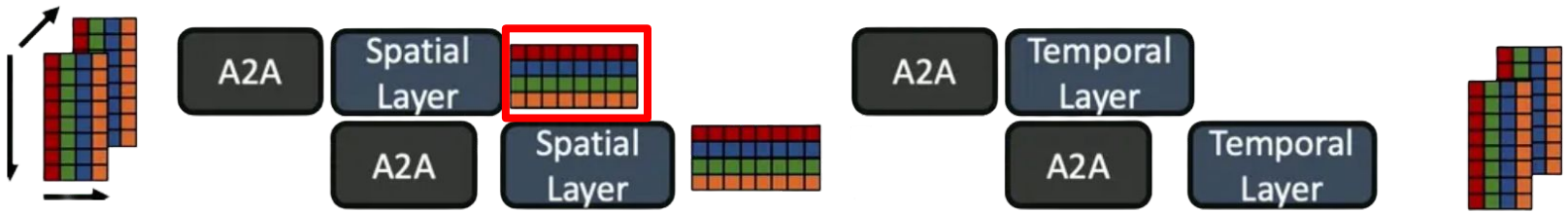
Inter-Layer Communication Scheduling

Solution: Split up the A2A's such that the last slice of a layer can send some data



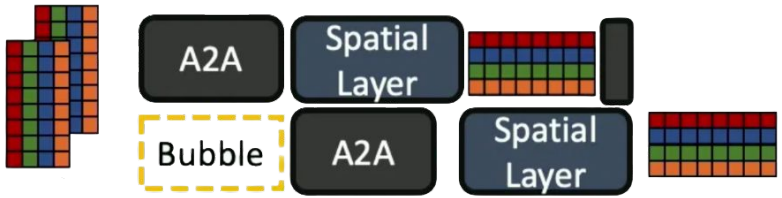
Inter-Layer Communication Scheduling

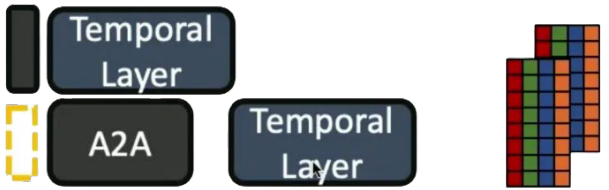
How can this be achieved?



Inter-Layer Communication Scheduling

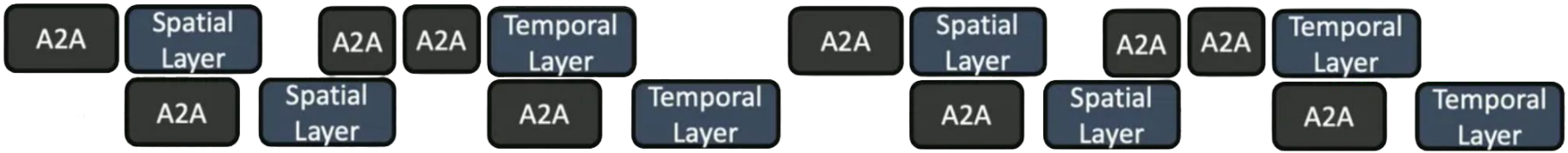
Once a slice's computation finishes, start a partial A2A





Inter-Layer Communication Scheduling

Scales between every layer switch between temporal and spatial



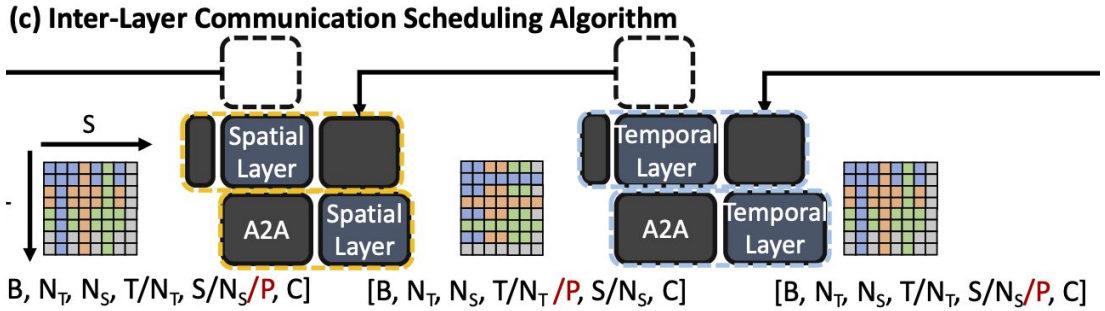
New Communication Latency: $(Comm_T + Comm_S)/N_S*N_T$

Inter-Layer Communication Scheduling

2 New Hyperparameters:
L_T and L_S

L_T : Number of A2A partitions lifted to overlap each temporal layer

L_S : Number of A2A partitions lifted to overlap each spatial layer



Experiment Setup

Baseline Comparisons of other ST-DiT Implementations:

- DeepSpeed Ulysses (used by OpenSora)
- Dynamic Sequence Parallelism (DSP)
- *Metrics gathered with the NVIDIA Nsight System*

Hardware:

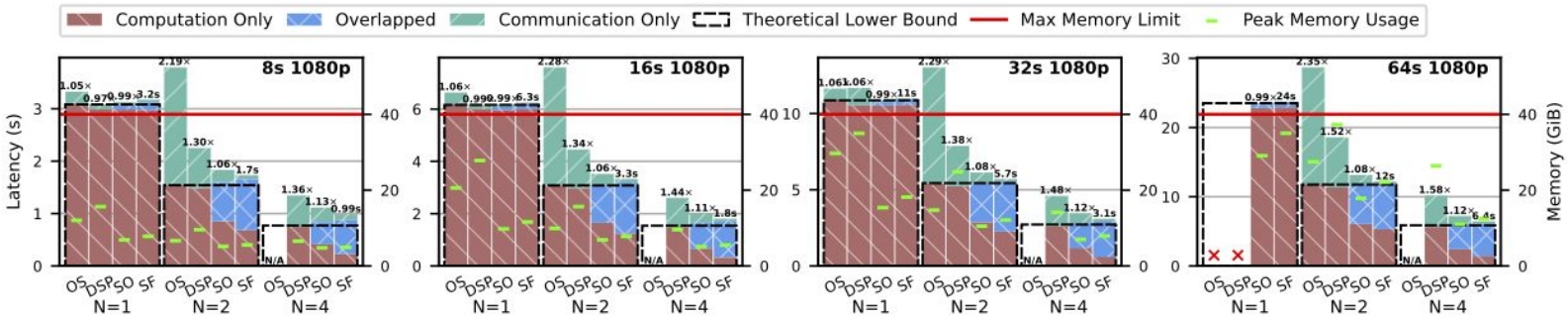
- AWS p4d.24xlarge machines (N=1, 2, 4)
  - 8 A100 GPUs (40 GiB VRAM)
  - Connected by NVSwitch & 400Gbps cross-machine AWS Elastic fabric

Hyperparameters:

- $N_T = N_S = 4$
- $L_T = 1$ $L_S = 3$

End-To-End Performance Evaluation

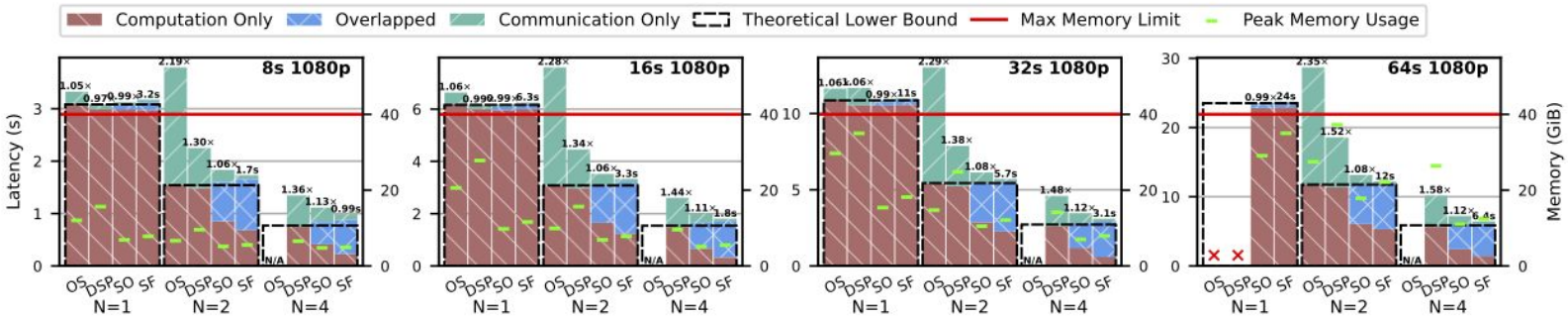
- OpenSora and DSP found to have high communication overheads
- ScaleFusion achieves an average:
  - 1.32x (up to 1.52x) speedup on 2 machines
  - 1.4x (up to 1.58x) speedup on 4 machines
- **Strong Scaling**
  - 1.93x in 2 machines
  - 3.60x in 4 machines



End-To-End Performance Evaluation

Comparing Intra and Inter Layer Communication Algorithms

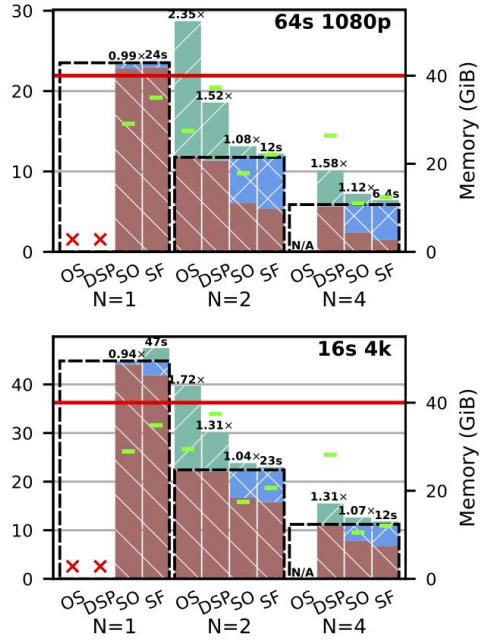
- Intra Only (**SO**)
  - 1.26x Speedup (up to 1.41x)
- Inter On Top of Intra (**SF**)
  - Additional 1.08x Speedup (up to 1.13x)



End-To-End Performance Evaluation

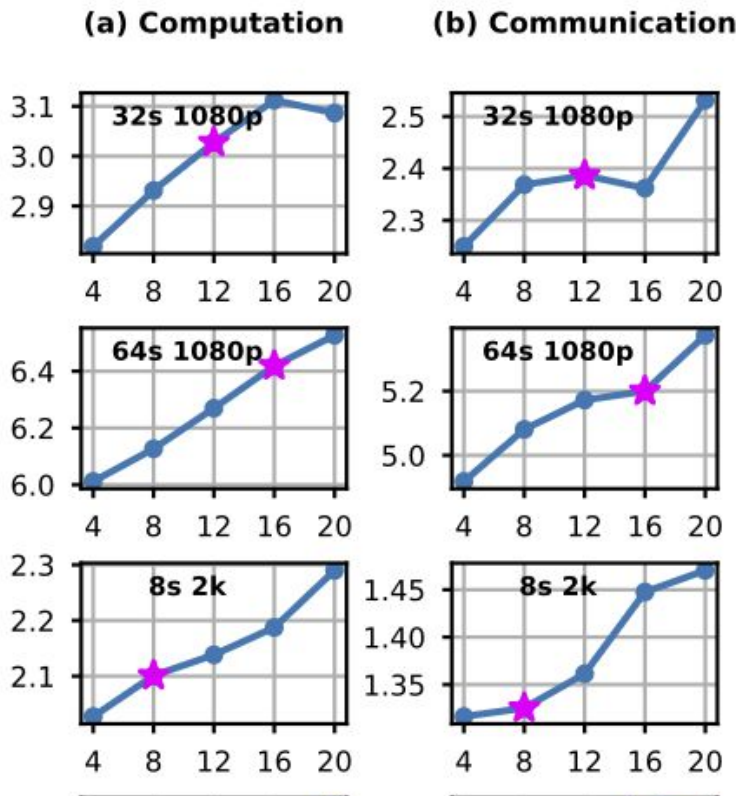
Peak Memory Usage

- Both OpenSora and DSP suffer from Out Of Memory (OOM) errors from generating high resolution or long videos
- ScaleFusion Memory Usage Reduction:
  - 1.28x reduction (up to 1.43x) compared to OpenSora
  - 1.87x reduction (up to 2.33x) compared to DSP



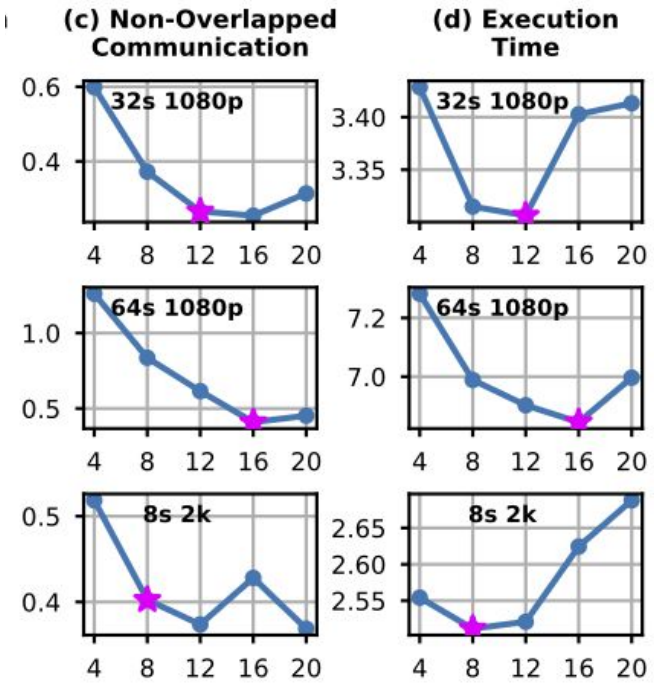
Hyperparameter Study

- **N_T and N_S**
  - More slices increases compute overhead
  - Communication latency increases with more A2A calls



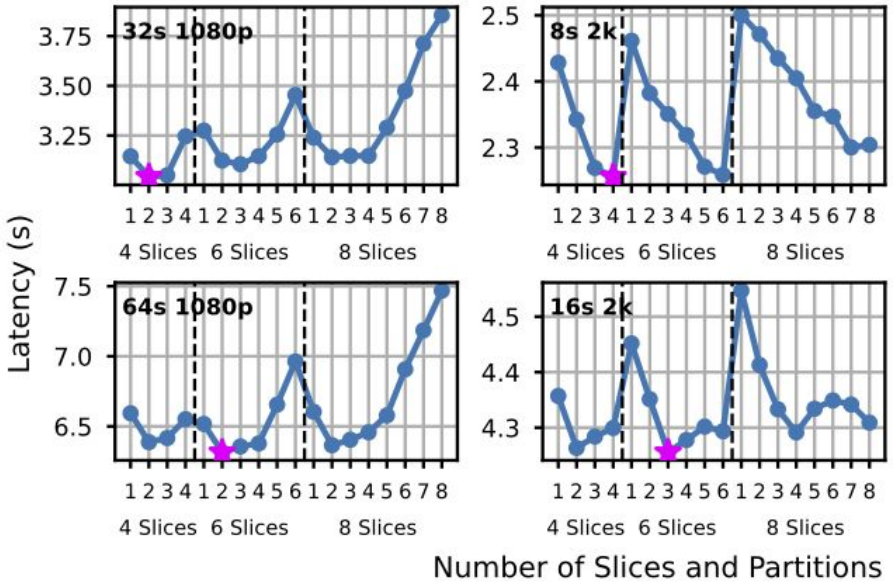
Hyperparameter Study

- **N_T and N_S**
  - More slices decreases non-overlapped communication (first A2A decreases)
- ***Concluded Default***
  - N_T=N_S= 4
- Best non-default only yields 2.7% speedup (on average)



Hyperparameter Study

- **L_T and L_S**
  - L_T always equal to 1
  - L_S varied as execution times vary across different experiment configurations
- ***Concluded Default***
  - L_T= 1
  - L_S= 3
- Best non-default only yields 0.7% speedup (on average)



ScaleFusion Limitations

- **ScaleFusion is architecture specific :** models with different attention structure (e.g. Full 3D global attention) see less gains.
- Intra-layer pipelining is sensitive to slices N_T and N_S splice parameters.
  - Too few slices > weak overlap, Too many > inefficient compute
  - Needs to be fine tuned per hardware/workload
- Inter-layer scheduling is most beneficial when there is enough compute slack to hide communication
- Total communication volume isn't reduced, its latency is just hidden, unlike DeepSpeed-Ulysses

Fast-dLLM & ScaleFusion: Two Ways to Accelerate Diffusion Inference

Fast-dLLM (Diffusion LLMs)	ScaleFusion
Model: Diffusion LLMs	**Model:** ST-DiT video diffusion
Bottlenecks: no KV cache & poor parallel decoding quality	**Bottleneck**: cross-machine A2A & back-to-back deps (comm can be 30–50% on multi-machine)
Solution: ● block-wise approximate KV cache (+DualCache) ● confidence-aware parallel decoding to avoid dependency errors	**Solution**: ● intra-layer pipelining & inter-layer lifting using spatial-temporal independence
Effect: large throughput gains (up to ×27.6)	**Effect:** better multi-machine scaling & speedup

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Thank you!